Student notes — Session 7 · Functions

Slides: [`Class_07_Functions.pdf`](../slides/Class_07_Functions.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Functions in C

The programs of this class are on Blackboard: Week 4 module, *Class 7 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Until today, every program was one `main`. Today you learn to cut a program into small named pieces: **functions**.

Quiz 2 is on Friday 2 October and covers classes 3 to 6. Functions are **not** on Quiz 2. They are on Quiz 3.

Slide 2 — What the loops gave us

What you need from last week:

- A loop has four parts: the **start**, the **check**, the **body** and the **update**.
- A total starts at 0 and is made **before** the loop: `sum += i;` inside it.
- A counter goes up by one inside an `if`: `count++;`.
- To check input, ask again while it is bad: `while (bad) { ... scanf ... }`.
- A product starts at 1: `fact *= i;`.

Slide 3 — Recap: what is on the screen?

Try to answer before you look at the next slide. `i` goes from 1 to 10. `count` goes up when `i % 4 == 0`, which means: `i` divides by 4 with nothing left over.

Slide 4 — Recap: what is on the screen?

It prints `2`. From 1 to 10, only 4 and 8 divide by 4.

Slide 5 — Today's roadmap

Six steps today:

1. Why functions.
2. How to write a function and call it.
3. Parameters, `void` and `return`.
4. Loops and decisions inside a function.
5. One whole program, split into functions.
6. The common bugs, and then you write two functions.

Slide 6 — Why functions

Part one: why we need functions. The short answer: so you can write a piece of program **once** and use it many times.

Slide 7 — Three rows of stars, copied

This program prints three rows of stars, 3, 5 and 7 long. The same `for` loop is written three times. Only the number is different. File: `02_stars_copy.c`.

Slide 8 — What if one copy must change?

Imagine you now want `#` instead of `*`. You must change three places. If you forget one, the rows disagree. With ten rows, ten places.

Copied code is a bug waiting to happen. The better way: write the loop **once**, and give it a **name**.

Slide 9 — The same program, with a function

This program prints exactly the same three rows. The loop is now inside a function called `print_stars`. File: `03_stars_function.c`.

Slide 10 — Written once: print_stars

The top part is the function. The loop is written here, once. `n` is the number of stars to print. The word `void` means: this function gives nothing back; it only prints. You will learn every part of the first line in a few slides.

Slide 11 — Used three times

The bottom part is `main`. Each line like `print_stars(3);` is a **call**: it tells the computer "run `print_stars` now, and this time `n` is 3".

To change `*` to `#` you now change **one** place, inside `print_stars`, and every row changes.

Slide 12 — You have used functions since class 1

You have used functions from the first day. `printf`, `scanf`, `sqrt` and `pow` are all functions. Someone wrote them once, in a library. You use them by writing their name and giving them values in brackets, like `sqrt(16)`.

Today you write your own.

Slide 13 — Writing and calling a function

Part two: how to write a function and how to use it.

Slide 14 — Our first function: square.c

Here is the whole program. File: `04_square.c`.

```
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int y = square(5);
    printf("square(5) = %d\n", y);
    printf("square(7) = %d\n", square(7));
    return 0;
}
```

It prints:

```
square(5) = 25
square(7) = 49
```

The function `square` is written **above** `main`, outside it. It is not inside `main`'s braces.

Slide 15 — Four parts of a function

Every function has four parts:

Part	In our example	What it means
type	`int`	the kind of answer the function gives back
name	`square`	the name you use to call it
parameter	`int x`	a box that gets the value you send in
body	`{ return x * x; }`	the work, between the braces

Slide 16 — Part 1: the type of the answer

The first word, `int`, says what kind of answer the function gives back. `square` gives back a whole number, so it says `int`. A function that gives back a number with a decimal point says `double`.

Slide 17 — Part 2: the name

You choose the name. The rules are the same as for variable names: letters, digits and _, no spaces, and it cannot start with a digit. Choose a name that says what the function does.

Slide 18 — Part 3: the parameter

In the brackets: `int x`. This is the **parameter**. It is a box that belongs to the function. It has a type and a name, like any variable. Every time the function is called, the value in the call is put into this box.

Slide 19 — Part 4: the body and return

The body is the part between `{` and `}`. It does the work. `return x * x;` means: work out `x * x` and hand that value back to whoever called the function.

Slide 20 — The call

In `main`, `square(5)` is a **call**. It is the name of the function and a value in brackets. The function runs with `x` equal to 5, and the answer, 25, goes into `y`.

Slide 21 — A call can sit inside printf

You do not need a variable to hold the answer. You can put the call straight into `printf`: `square(7)` becomes 49, and `printf` prints 49.

Slide 22 — Writing a function does nothing by itself

Writing a function does not run it. A function is like a recipe: it only runs when it is **called**. A program always starts in `main`, even when the functions are written above it.

Slide 23 — What happens at a call?

This version of `square` has `printf` lines inside, so you can see when each part runs. File: `05_call_flow.c`. Before you look further, write down the four lines you think it prints, in order.

Slide 24 — Step 1: main starts

The program starts in `main`. The first line printed is `main: before the call`.

Slide 25 — Step 2: the call jumps into square

At the call `square(5)`, the computer jumps into `square`. The parameter `x` gets a **copy** of 5. Remember the word "copy": it is important in class 8.

Slide 26 — Step 3: the body of square runs

The body of `square` runs. It prints `square got 5`, works out 25, and prints `square gives back 25`. While this happens, `main` waits.

Slide 27 — Step 4: return jumps back

`return result;` ends the function and jumps back to the line in `main` where the call was. The call `square(5)` is **replaced** by the answer, 25. So that line becomes `int y = 25;`.

Slide 28 — Step 5: main goes on

`main` goes on and prints `main: got 25`. The four lines are:

```
main: before the call
  square got 5
  square gives back 25
main: got 25
```

Slide 29 — A call, in four steps

What happens at every call:

1. The value in brackets is **copied** into the parameter.
2. The body of the function runs. The caller waits.
3. `return` sends **one** value back.
4. The call is **replaced** by that value, and the caller goes on.

Slide 30 — Predict: calls inside calls

Try to work out the three lines before you look at the answer. `a` is 3. File: `06_several_calls.c`.

Slide 31 — Predict: calls inside calls

The three lines are `25`, `16` and `16`.

- `square(a) + square(4)` is 9 + 16 = 25.
- `square(square(2))`: first `square(2)` is 4, then `square(4)` is 16.
- `square(a + 1)`: first `a + 1` is worked out, which is 4, then `square(4)` is 16.

The value in the brackets is always worked out **before** the function runs.

Slide 32 — Predict: square below main

Here the function `square` is written **below** `main`. What happens when you compile? File: `07_bug_after_main.c`.

Slide 33 — gcc stops: it has not met square yet

It does not compile. gcc says:

```
error: implicit declaration of function 'square'
```

gcc reads your file from the top down. When it reaches the call in `main`, it has never seen `square`. (With older compilers, such as gcc 13, this was only a warning. With the course compiler it is an error.)

Slide 34 — Fix 1: the function above main

The first fix: write the whole function **above** `main`. This is what we do in this course.

Slide 35 — Fix 2: a prototype above main

The second fix: leave the function below `main`, but write its first line above `main`, ending with a semicolon:

```
int square(int x);
```

This line is called a **prototype**. It tells gcc: "a function called `square` exists; it takes an `int` and gives back an `int`; its body comes later". File: `08_prototype.c`. You will see this in books. Both fixes are correct.

Slide 36 — Parameters, `void` and `return`

Part three: functions with more than one value in, functions that give nothing back, and `return`.

Slide 37 — Two parameters

A function can have several parameters, separated by commas: `int add(int a, int b)`. The values in a call are matched by **position**: the first value goes to `a`, the second to `b`. `add(2, 3)` is 5. File: `09_add.c`.

Slide 38 — Predict: the order of the values

`power(base, exp)` multiplies `base` by itself `exp` times. Inside it there is a loop, and `result` starts at 1, like the factorial of class 6. What are `power(2, 3)` and `power(3, 2)`?

Slide 39 — The order matters

`power(2, 3)` is 2 × 2 × 2 = 8. `power(3, 2)` is 3 × 3 = 9. The same two numbers in a different order give a different answer. The compiler cannot catch this mistake, because both values are `int`. File: `10_power.c`.

Slide 40 — A loop inside a function

A function can have its own loop and its own variables, like `result` here. `main` cannot see them. `main` only gets the answer.

`power(10, 0)` is 1: the loop runs zero times, so `result` keeps its start value, 1.

Slide 41 — Predict: the average of 7 and 8

Two functions that work out an average. One works with `int`, the other with `double`. Both are given 7 and 8. What does each give back?

Slide 42 — int or double: integer division again

```
int version: 7
double version: 7.50
```

In the `int` version, 7 + 8 = 15 and 15 / 2 is 7: integer division throws away the `.5`, as you saw in class 2. The `double` version keeps it. If the answer can have a decimal point, the function's type must be `double`. File: `11_average.c`.

Slide 43 — void: a function with no answer

`void` in front of a function name means: this function gives **nothing** back. It does a job, such as printing, and that is all. `print_stars` from the start of the class is a `void` function.

Slide 44 — void with no parameters

`void greet(void)` gives nothing back and takes nothing in. It prints two lines every time it is called. When you call it, you still write the brackets: `greet();`. File: `12_greet.c`.

Slide 45 — Printing is not returning

Two functions. `square_print` **prints** the square. `square` **returns** it. What does the program print? File: `13_print_vs_return.c`.

Slide 46 — Printing is not returning

It prints `25` (printed by `square_print` itself) and then `total = 25`. `square` handed back 9 and 16, so `main` could add them.

Slide 47 — You cannot add two printouts

If you try to add the results of `square_print`, the program does not compile:

```
error: void value not ignored as it ought to be
```

In plain words: this function gives back nothing, so there is nothing to add. File: `14_bug_void_value.c`.

Slide 48 — Print or return?

- **Print:** the value appears on the screen. The caller gets nothing.
- **Return:** the value is handed back. The caller can use it: add it, store it, print it.

Most functions should **return** their answer and let `main` decide what to print.

Slide 49 — return ends the function at once

`max` gives back the larger of two numbers. When `a > b`, `return a;` runs, and the function **ends at once**. Nothing after it in the function runs. File: `15_max.c`.

Slide 50 — Otherwise: the second return

The second `return b;` is only reached when `a > b` was false. That is why no `else` is needed. `max(3, 9)` and `max(9, 3)` both give 9.

Slide 51 — Predict: a function that calls a function

`max3` finds the largest of three numbers by calling `max` twice. What is `max3(4, 11, 7)`?

Slide 52 — Predict: a function that calls a function

1. First `max(4, 11)` is 11. Then `max(11, 7)` is 11. A function can call another function.

Slide 53 — Yes or no: return 1 or 0

`is_even` answers a yes-or-no question: it gives back 1 for yes and 0 for no. In C, 1 means true and 0 means false. File: `16_is_even.c`.

Slide 54 — The call goes straight into an if

Because the answer is 1 or 0, you can put the call straight into an `if`: `if (is_even(i))`. The program prints `1 is odd`, `2 is even`, and so on up to `5 is odd`.

Slide 55 — Loops and decisions inside a function

Part four: the loops of class 6 and the `else if` of class 4, inside functions.

Slide 56 — Factorial as a function

The factorial of class 6, now as a function `factorial(n)`. `main` calls it for every `n` from 0 to 10 and prints `0! = 1`, `1! = 1`, `2! = 2`, up to `10! = 3628800`. File: `17_factorial.c`.

Slide 57 — Why is 0! equal to 1?

Call `factorial(0)`. How many times does the loop run? What comes back?

Slide 58 — Why is 0! equal to 1?

The loop runs zero times, because `2 <= 0` is false at the very start. So `result` is still 1, and 1 is returned. In mathematics too, 0! is 1.

Slide 59 — A letter grade: char letter(int score)

`letter` takes a score and gives back a letter grade. Its type is `char`, because it gives back one character. Inside is an `else if` chain like the ones in class 4; each branch returns at once. (The limits here are only an example; they are not ADA's grading scale.) File: `18_letter.c`.

Slide 60 — Using letter

To print a `char`, use `%c`. The program prints `95 -> A`, `72 -> C` and `40 -> F`.

Slide 61 — The input check of class 6, as a function

The input check of class 6, as a function. `read_between(low, high)` reads a number, keeps asking while it is below `low` or above `high`, and then **returns** the good number. File: `19_read_between.c`.

Slide 62 — One function, two different checks

The same function does two different checks: a month from 1 to 12 and a day from 1 to 31. An example run, where the user types 13, 0, 7, then 40, 25:

```
Month: 13
Type 1 to 12: 0
Type 1 to 12: 7
Day: 40
Type 1 to 31: 25
Date: 25/7
```

Slide 63 — One program, split into functions

Part five: a whole program, split into functions so that `main` reads like a list of steps.

Slide 64 — The job

The program: for three students, read a score from 0 to 100 (and ask again if the score is not in that range), work out the letter grade, and print one report line.

Slide 65 — You could write it all in main

You could write the whole program inside `main`. It works: it is `21_report_all_in_main.c` in the class code. But it is 27 lines in one block, with reading, checking, grading and printing mixed together. To change one part, you first have to find it.

Slide 66 — Split: main is a list of steps

When the work is split into functions, `main` is short:

```
int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("Score of student %d: ", i);
        int score = read_between(0, 100);
        print_report(i, score);
    }
    return 0;
}
```

Read it as steps: ask, read a good score, print the report. File: `20_report.c`.

Slide 67 — Step 1: read_between

The first function is `read_between`, from slide 61. In class we copied it from `19_read_between.c`.

Slide 68 — Step 2: letter

The second function is `letter`, from slide 59, copied from `18_letter.c`. Writing a function once and using it in a second program is called **reuse**.

Slide 69 — Step 3: print_report

The third function is new: `print_report(n, score)` prints one report line. Its second `printf` calls `letter(score)`. A function calling another function.

All three functions are written **above** `main`.

Slide 70 — Run it

With the scores 95, 72 and 40:

```
Score of student 1: 95
Student 1: 95 points, grade A
Score of student 2: 72
Student 2: 72 points, grade C
Score of student 3: 40
Student 3: 40 points, grade F
```

Slide 71 — A bad score is caught inside read_between

If you type 120, `read_between` asks again (`Type 0 to 100: `). `main` never sees the bad score. Then 88 gives grade B.

Slide 72 — Why split a program?

Four reasons to split a program into functions:

- **Read:** `main` reads like a list of steps.
- **Change:** new grade limits? You change one function.
- **Test:** you can try `letter(89)` on its own.
- **Reuse:** `read_between` works for months, days and scores.

Slide 73 — Common bugs

Part six: the mistakes everybody makes with functions. All of them were compiled with the course compiler (gcc 16.2).

Slide 74 — Five function bugs

1. A missing `return`.
2. The wrong number of values in a call.
3. A `;` after the first line of the function.
4. A call without `()`.
5. A function used before gcc has seen it (slide 33).

Slide 75 — Predict: one return is missing

In this `max`, `return b;` is missing. When `a > b` is false, the function reaches its closing brace with no `return`. What does `max(3, 9)` print? File: `22_bug_no_return.c`.

Slide 76 — A missing return: a warning, then a wrong answer

gcc with `-Wall` warns:

```
warning: control reaches end of non-void function
```

But the program is built, and it runs. On our computer it printed `max(3, 9) = 3`, which is wrong. On another computer it may print a different wrong number. A warning often means the answer will be wrong: always read your warnings.

Slide 77 — The wrong number of values

`square` takes one value but was given two; `add` takes two but was given one. Both are errors, and gcc says what it expected:

```
error: too many arguments to function 'square'; expected 1, have 2
error: too few arguments to function 'add'; expected 2, have 1
```

(An **argument** is the value you put in the brackets of a call.) File: `23_bug_wrong_args.c`.

Slide 78 — A semicolon after the first line

With a `;` after `int square(int x)`, that line becomes a prototype, and the `{` after it belongs to nothing. gcc says `expected identifier or '(' before '{' token`. The error points at the `{`, but the mistake is the `;` on the line above. File: `24_bug_semicolon_header.c`.

Slide 79 — A call without brackets

`greet;` without brackets names the function but does not call it. gcc warns `statement with no effect`, and `Hello!` is printed only once. To call a function, you need the brackets: `greet();`. File: `25_bug_no_brackets.c`.

Slide 80 — What gcc says

Bug	What gcc 16.2 says	Is a program built?
no `return`	warning: control reaches end of non-void function	yes, and it runs wrong
wrong number of values	error: too many / too few arguments	no
`;` after the first line	error: expected identifier or '(' before '{' token	no
no `()` in the call	warning: statement with no effect	yes, and it runs wrong
used before it is seen	error: implicit declaration of function	no

The warnings are the dangerous ones: you get a program that runs wrong. Always compile with `-Wall` and read what it says.

Slide 81 — Now you write

Part seven: now you write functions yourself.

Slide 82 — Four small functions

Four tasks. In each one, write the function **above** `main`, then call it from `main`.

1. `int cube(int x)`: gives back `x * x * x`.
2. `int sign(int n)`: gives back -1, 0 or 1.
3. `void print_rectangle(int width, int height)`: prints a rectangle of stars.
4. `double celsius_to_fahrenheit(double c)`.

Tasks 1 and 2 were done in class; 3 and 4 are homework. Solutions: files 26 to 29 of the class code. Try first.

Slide 83 — Task 1: cube

Write `int cube(int x)`. In `main`, read a number and print its cube. Example run:

```
Number: 3
Cube: 27
```

Test it with -2 (the cube is -8) and with 0.

Slide 84 — Task 2: sign

Write `int sign(int n)`. It gives back -1 when `n` is negative, 0 when `n` is zero, and 1 when `n` is positive. Example run:

```
Number: -7
Sign: -1
```

Slide 85 — Solutions: cube and sign

One possible solution of each. Yours may look different and still be right. Always test with a negative number and with 0.

Slide 86 — Task 3, at home: a rectangle

Write `void print_rectangle(int width, int height)`. Hint: copy `print_stars` from `03_stars_function.c` into your file, and call it once for every row, with a loop. `print_rectangle(6, 3)` prints three rows of six stars. Solution: `28_task_rectangle.c`.

Slide 87 — Task 4, at home: Celsius to Fahrenheit

Write `double celsius_to_fahrenheit(double c)`, which gives back `c * 9 / 5 + 32`. In `main`, read the temperature with `scanf("%lf", &c)` and print the answer with `%.1f`. 100 gives 212.0, 0 gives 32.0. Try -40. Solution: `29_task_celsius.c`.

Slide 88 — Class 7 summary

- A function has a **type**, a **name**, **parameters** and a **body**.
- A call copies the values in, runs the body, and gets **one** value back.
- Write the function **above** `main`, or give a prototype above `main`.
- `void`: the function gives nothing back. It only does a job.
- `return` ends the function at once.
- Split a program so that `main` reads like a list of steps.

Slide 89 — Project 1 is out

Project 1: Simplified Blackjack. You play a simple card game against the computer: get as close to 21 as you can without going over. It uses loops, decisions and functions. The brief is on Blackboard: the exact rules, example runs, and how it is marked.

Due: Sunday 11 October 2026, 23:59. Start this week. After you submit, you may be asked to explain your code and make a small change to it, so write it yourself.

Slide 90 — Before Friday

- **Knowledge check 7** on Blackboard covers today's slides.
- Do **Tasks 3 and 4**.
- **Quiz 2: Friday 2 October**, first 25 minutes, on paper, no laptop. It covers classes 3 to 6: data types, conditions, `switch` and loops. Not functions. Come on time.
- Today's code: Blackboard, Week 4 module, *Class 7 code*.

On Friday, after the quiz: where variables live, why a function cannot change your variables, and a function that calls itself.